

Agent Substitution v0 — Results Report

Elena (PM) · 2026-06-12 · pilot #1: agent substitution (Upstream: v0-team-report-zh.md = the plan before running; this = the results after running. Every number was audited against the run data — see §3.)

0. Bottom line

On real agentic tasks from a public benchmark (ALE), we produced the first quality × cost table. Conclusion: no cheap model matches Claude on quality (best is GLM-4.6 ≈ 76%), but GLM-4.6 is the best candidate on *both* axes — 76% of Claude's quality at ~8× lower cost — enough to support a hybrid routing strategy (cheap model handles what it can, Claude handles the hard tasks + final review). v0 quantifies "who can be replaced, how much is saved, when to fall back" — exactly the layer BFC's gateway "Smart Cost Routing" is missing.

1. Result: quality × cost table

Main table (apples-to-apples: the 14 tasks all 4 models completed — same set, directly comparable)

Model	Quality (mean score)	vs Claude quality	Cost (\$/task)	Cheaper
Claude sonnet-4.6 (baseline)	0.653	100%	\$0.439	1×
GLM-4.6	0.497	76%	\$0.052	8.4×
DeepSeek V3.2	0.433	66%	\$0.162	2.7×
Qwen3-Coder	0.377	58%	\$0.209	2.1×
Kimi K2	— [routing unusable, see §2③]	—	—	—

Notes on methodology: – **Quality** = ALE `ale_run` auto-scoring, per-run (deterministic scoring primarily; some tasks use an LLM judge). Same task set, comparable across models. – **Cost** = each run's *real* token counts × that model's *real* OpenRouter price (cache_read ×0.1, cache_create ×1.25) — **not** the harness's Claude-priced estimate (which overstates cheap-model cost ~7–14×, see §3). – All models use the fixed `claude_code` harness; only `model:` is swapped — a clean single-variable comparison. – **Why 14 common tasks:** completion counts differ (Claude 18 / DeepSeek 16 / GLM 17 / Qwen 17) and the failed tasks differ per model; a fair comparison uses only tasks every model finished. Full-set numbers are in the appendix.

2. Key findings

- ① **No cheap model matches Claude's quality (best is GLM $\approx$ 76%).** On real agentic tasks (open a terminal, run tools, produce verifiable deliverables) cheap models are clearly weaker. So v0 does **not** support "swap Claude out wholesale for one cheap model" — it supports **hybrid**.
- ② **GLM-4.6 is the only candidate that wins on both axes: 76% quality (highest among candidates) + ~8x cheaper.** The cost advantage's secret is **token efficiency**, not the price tag: on the same task (pe_screening, where both models actually produced results), GLM scored higher (0.944 vs 0.869) using only ~126K input tokens vs DeepSeek's ~1.76M (13.9x) — in dollars, \$0.09 (GLM) vs \$0.41 (DeepSeek). The key: **DeepSeek's input price is actually lower (\$0.23 vs GLM's \$0.43/M), yet token bloat makes it 4x more expensive per task.** So at the full-set level DeepSeek's lower unit price still yields only 2.7x savings; GLM yields 8x+. **"Cheap" = token efficiency $\times$ unit price — you can't read it off the price tag alone.**
- ③ **Kimi K2 is unusable via OpenRouter + claude_code (directly relevant to the gateway).** After fixing throttling, kimi still hit provider-side **400 invalid request** errors; the runs that did complete scored ~0 (full-set mean 0.053). Takeaway: **not every "cheap model" routes cleanly through a gateway's Anthropic-compatible endpoint to drive an agent harness** — precisely the pitfall BFC's "Smart Cost Routing" should screen out for customers (verify "this model $\times$ this harness actually runs" *before* routing to it).
- ④ **Methodology lesson (only surfaced in the audit, worth remembering):** model comparison must be done on the set of tasks all models completed. If you use "each model's own completed set," DeepSeek *appears* to lead (66% > GLM 63%) — because it **skipped two hard tasks it couldn't do** and got an invisible bonus. On the same 14 tasks all models finished, GLM (76%) is the true leader. **The reporting convention flips the conclusion — something to be explicit about externally.**
- ⑤ **Strategic implication: hybrid routing + "verify-it-runs, then route by value."** Cheap model (GLM-4.6 first) handles what it can; Claude takes the hard tasks + final review. This v0 table is the prototype of that routing's "upstream classifier" — ALE's methodology does the offline tiering, BFC's real traffic refines it online.

3. Credibility: how the numbers were derived + the audit

- **Cost method validated:** the token $\times$ price formula reproduces Claude's per-task cost (\$0.86) — **exactly matching** the harness's self-reported total_cost_usd $\checkmark$. Cheap models use the same formula with their own real prices.
- **Why not the harness's self-reported cost:** it prices every model at Claude's rate, overstating cheap-model cost 7–14x (measured: DeepSeek 13.9x, Qwen 13.4x, GLM 7.3x) — so it's discarded in favor of the recomputed cost.
- **Audited line by line:** every quality score, cost, and token count was traced back to a specific run and recomputed; the audit corrected three things (DeepSeek's false lead, one bad cost reconciliation, a few rounded multiples) before this table was finalized.
- **Actual OpenRouter bill = \$67.35** (includes all demo / smoke / validation / retry runs across the build, not just the main table) — the main-table runs are only part of it; the bill is higher because of build-

and-retry overhead (see §4).

- **Every number traces back** to a run record in the repo's `run-log.md`.

4. Engineering evidence: real problems found by running (not imagined from the docs)

- ★ **Found and fixed a reliability bug in ALE itself:** its zone-capacity error classifier misses GCP's most common capacity error (YAML line-wrap + an error-code substring mismatch), so VM cross-zone fallback **never fires** and a single zone's stockout fails the whole run. Root-caused, fixed locally, unit-tested. (See repo `findings.md`; whether to report it to Han is TBD by the team.)
- **Fresh-project GCP quota walls:** the 600GB pd-ssd/hyperdisk image exceeds a new project's default quotas (SSD/HDB 500GB each), resolved by requesting increases.
- **8-vCPU capacity varies with load** (observed: stocked out in the evening, available in the morning) → run real-task sweeps off-peak.
- None of these affect scoring / model comparison (pure infrastructure-retry layer).

5. Limitations

- **ALE's distribution ≠ our real traffic.** v0 is a "capability tiering + short-list" on a public benchmark — **not** proof of "replaceable on BFC customers' real agent traffic," which needs real usage data to follow. Externally, label everything "measured on a public benchmark."
- **Small sample:** common set N=14 (full set N=16–18); near-term scores are inherently noisy — **the trend is reliable; absolute values await a larger N.**
- **Task set:** Linux (cpu-free-ubuntu) near-term tier only; Windows / GPU / larger N left to expand.
- **Kimi data is not used for conclusions** (routing unusable).

6. Decision points for the team / Paul

1. **Lock the bar (substitution threshold) — reference solution ready, Paul to confirm or adjust:**

Framework: a cheap model "qualifies / makes the short-list" if quality $\geq X\%$ of Claude AND cost $\leq 1/Y$ of Claude — both must pass.

Three reference tiers applied to this table (14 common tasks):

Tier	Quality bar X	Cost bar 1/Y	Passes	Read
Conservative (low risk)	$\geq 85\%$	$\leq 1/2$	(none)	current cheap models insufficient; need stronger models or per-task routing
Default (recommended)	$\geq 70\%$	$\leq 1/3$	GLM-4.6 only	GLM makes the short-list, handles what it can, Claude as fallback
Aggressive (max savings)	$\geq 60\%$	$\leq 1/2$	GLM-4.6 + DeepSeek	saves more, but more fallbacks and more quality variance

- **Why default 70% / 3x:** GLM passes **both** (76%, 8.4x); DeepSeek (66%) / Qwen (58%) fail quality — a clean short-list of just GLM, easiest to decide and to defend externally.
 - **This is only a coarse screen (who makes the short-list).** Real hybrid routing is **per-task / per-task-type** — "cheap by default, fall back to Claude on tasks it fails" — not aggregate means. v0 already stores per-task scores, so **v0.1 can estimate the actual blended saving** and refine X/Y into a per-task policy.
 - **Paul to decide:** adopt default 70% / 3x ($\rightarrow$ short-list = GLM), or slide up (conservative) / down (aggressive) by acceptable business risk?
2. **Expand scope:** whether to add Windows/GPU tasks, increase N, add candidates (incl. re-testing Kimi via its native endpoint).
3. **Cost:** the full v0 run = **OpenRouter \$67 + GCP ~\$10–13 (estimated, billing lags) $\approx$ \$77–80**, all infrastructure cost for a company pilot, itemized — to be reimbursed.

7. Next steps (who does what)

Now · this week

- **Paul (decision, blocks everything downstream):** lock the bar — reference solution in §6.1 (recommended default 70% / 3x $\rightarrow$ short-list = GLM, with conservative/aggressive tiers also prepared), confirm or adjust. This is the prerequisite for the short-list / routing recommendation; until it's set, we stay at "data" and can't reach "recommendation."
- **Elena (PM):** ① finalize v0 delivery (this report + repo: run-log evidence / findings bug writeup / qualityxcost table); ② the moment Paul locks the bar, apply it and produce v0.1 "short-list + routing recommendation" (who can replace, when to fall back).

Next · v0.1 and expansion (PM-led, needs teammates)

- **Elena (PM):** ① expand the task set — add Windows/GPU tasks, increase N to cut noise (current common set N=14 is small); ② re-test Kimi via its native endpoint to isolate OpenRouter-routing vs the model itself; ③ produce an English report version as needed.
- **Han (ALE author), support:** ① confirm whether the per-run scoring convention is the one we used; ② the future scope of ALE's private task set available to us.

- **Abby, support:** ① official pricing tables for the candidate models, to check our recomputed costs; ② whether task_type can be inferred from tokens / channel; ③ whether any success/failure signal exists online — to pave the way for the real-traffic phase.

Real-traffic phase · handoff

- **Elena x SDE:** align the instrumentation fields for live agent traffic (which task type, which model x harness, success or not, how much it cost), to move from "offline tiering on a public benchmark" to "online refinement on our own traffic" — the key leap from v0's "can it replace" to the product's "real-time routing."

Appendix: full-set numbers (N differs — for reference only, not a comparable conclusion)

Model	N	Full-set mean	Note
Claude sonnet-4.6	18	0.664	
DeepSeek V3.2	16	0.442	missing cp_test, recsys (failed)
GLM-4.6	17	0.419	missing nhanes (failed)
Qwen3-Coder	17	0.361	missing bpmn (failed)
Kimi K2	15	0.053	routing unusable, not reliable

⚠️ N differs and the missing tasks differ, so this is **not directly comparable across models** (DeepSeek looks inflated because it skipped hard tasks). All conclusions use the §1 main table (14 common tasks).

Data and full evidence trail: repo `agent-eval-v0` (run-log.md / findings.md).

Repo: <https://github.com/yingshill/agent-eval-v0>